Tokenizers and the hidden tax

How a vocabulary decides what a token costs in every language, why most models have too few of them, and the architecture choices — embedding size, tied weights, the softmax bill — that follow from the first decision you make

---

Take one paragraph — say, the opening of a news story — and run it through the tokenizer of a model you are considering. In English it comes back as about 180 tokens. The same paragraph, translated into Spanish, comes back as 240 or 260; into Portuguese, about the same; into Quechua, past 500. Nothing about the model has changed, and yet the Spanish user is paying 35–45% more compute for every sentence, getting 25–30% less effective context, and waiting longer per answer; and your pre-training corpus, which Chapter 10 reported as 4.6% Spanish by document count, is a different fraction once you count what the model actually sees. That is the hidden tax, and it is set by the one decision in the pipeline that is almost never revisited: the vocabulary. This chapter is about making it on purpose — measuring the tax, understanding where it comes from, choosing or adapting a vocabulary that lowers it, and accounting for the architectural consequences, because a vocabulary is also the largest matrix in a small model and the most expensive layer in every forward pass.

The chapter closes the data arc. Chapter 10 built the corpus and left one sentence hanging — "measure the share after tokenization, not in bytes" — and Chapter 11 repeated it; here is where the measurement is made, and where the data chapters' numbers get re-stated in the unit the model is trained in. The chapter also absorbs what an earlier plan of this book called "architecture choices," because the ones that matter most at the budget we care about — vocabulary size, embedding dimension, tied versus untied embeddings — are not separable from the tokenizer.

12.1 The tax, measured

Start with the two numbers that define the problem. The **fertility** of a tokenizer on a language is the average number of tokens per word; the **compression** is bytes of text per token. Both are properties of the pair (tokenizer, language), and the ratio of a language's fertility to English's is the tax. The 2023 study that put this on the agenda translated the same texts into many languages and tokenized them with the vocabularies of the day: the same content differed in token count by up to 15× across languages, with gaps above 4× even for byte- and character-level schemes that were supposed to be neutral, and the consequences followed directly — higher API cost, higher latency, and less room in the context window for the languages that already had the least data[1]. For the Romance languages the gap is smaller but systematic: under a 32K English-centric vocabulary (Llama 2's), Spanish and Portuguese typically run 1.4–1.7×

English; under the 128K–262K multilingual vocabularies of 2024–2025 (Llama 3, Qwen3, Gemma 3), roughly 1.2–1.4× — the exact ratio depends on the text, and the lab measures it on your own corpus; treat these as the typical ranges rather than constants[1].

What the tax costs is arithmetic you already have. Compute per sentence scales with tokens, $6N$ per token for training and $2N$ for inference (Chapter 8), so a 1.3× fertility ratio is a 1.3× training bill for the same content and a 1.3× serving bill forever. Context is a fixed token budget, so a 1.3× ratio turns an 8K window into a 6.2K window of text. And the data mix is counted in tokens: a Spanish document that is 30% more tokens than its English equivalent occupies 30% more of the model's attention — which sounds like good news for Spanish until you see that the extra tokens are *fragments*, subword pieces that each carry less meaning and that the model must learn to reassemble, which is why high-fertility languages learn more slowly per token, not faster. Figure 12.1 puts the costs on one axis.

A worked example makes the mix point concrete. Suppose Chapter 10's corpus is 95% English and 5% Spanish by document, documents of equal length, and the tokenizer's fertility is 1.0 for English and 1.4 for Spanish. The token shares are then $0.95 \times 1.0 = 0.95$ and $0.05 \times 1.4 = 0.07$, so Spanish is $0.07/1.02 \approx 6.9\%$ of what the model sees — not 5% — and a DoReMi weight or a stage schedule written for 5% is off by nearly 40% before training starts. Under a vocabulary where Spanish fertility is 1.15, the same corpus is 5.7% Spanish. The mix is a property of the pair (corpus, tokenizer), which is why this chapter had to come before the data numbers are final.

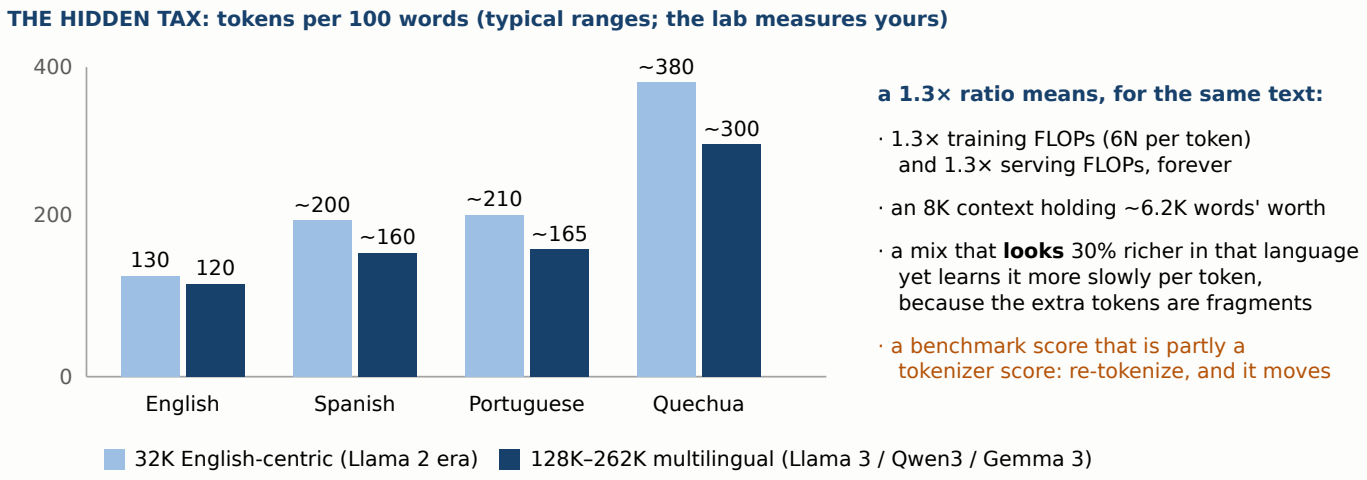


Figure 12.1 — Tokens per hundred words, by language and vocabulary generation. The bars are typical ranges for news-style text; the point is the *ratio*, which multiplies every cost in the book. Multilingual vocabularies narrow the gap for the Romance languages and leave it wide for languages with little training data.

```
# measure the tax on your own corpus: fertility and bytes/token per language, per
tokenizer
from transformers import AutoTokenizer
import collections, numpy as np
TOKS = {"llama2-32k": "meta-llama/Llama-2-7b-hf", "llama3-128k": "meta-
llama/Llama-3.1-8B",
        "qwen3-151k": "Qwen/Qwen3-8B", "gemma3-262k": "google/gemma-3-4b-pt"}
def tax(docs_by_lang, name):              # docs_by_lang: {"en": [...], "es":
[...], "pt": [...]}
    tok = AutoTokenizer.from_pretrained(TOKS[name])
```

```
    out = {}
    for lang, docs in docs_by_lang.items():
        n_tok = sum(len(tok(d, add_special_tokens=False).input_ids) for d in docs)
        n_words = sum(len(d.split()) for d in docs)
        n_bytes = sum(len(d.encode()) for d in docs)
        out[lang] = {"fertility": n_tok / n_words, "bytes_per_token": n_bytes /
n_tok}
    base = out["en"]["fertility"]
    for lang, m in out.items(): m["ratio_vs_en"] = m["fertility"] / base
    return out
# the mix re-stated in tokens: share_tokens[lang] = n_tok[lang] / sum(n_tok) — the
4.6% moves
```

12.2 Where tokens come from: BPE, mechanically

Every production tokenizer of the last five years is byte-pair encoding or a close cousin, and understanding it at the level of its two loops is what lets you predict the tax. BPE is trained once, on a sample of the corpus, and then applied deterministically. **Training** starts from a base alphabet — 256 byte values in byte-level BPE, so that any string is representable and there is no unknown token — and repeats one step: count all adjacent pairs of symbols across the corpus, merge the most frequent pair into a new symbol, add it to the vocabulary, until the vocabulary reaches its target size V. The order of merges is the tokenizer; applying it to new text means running the same merges in the same order. **Pre-tokenization** decides what the merges are allowed to span. The GPT-2 regular expression, inherited by most of its descendants, cuts text into words, numbers, punctuation runs, and leading spaces before BPE sees anything, so that a merge can never cross a word boundary — which is why "New York" is always two tokens, and why SuperBPE's one change (a second training stage with the whitespace restriction lifted, so that frequent multi-word expressions become single "superword" tokens) cuts token counts by up to 33% at a 200K vocabulary and, at fixed model size and compute, gained 4.0 points on average across 30 tasks and 8.2 on MMLU, with 27% less inference compute[2]. Digits are a special case: Llama 2 split every digit into its own token, Llama 3 groups up to three, and the choice changes arithmetic ability measurably because it changes what a "number" looks like to the model.

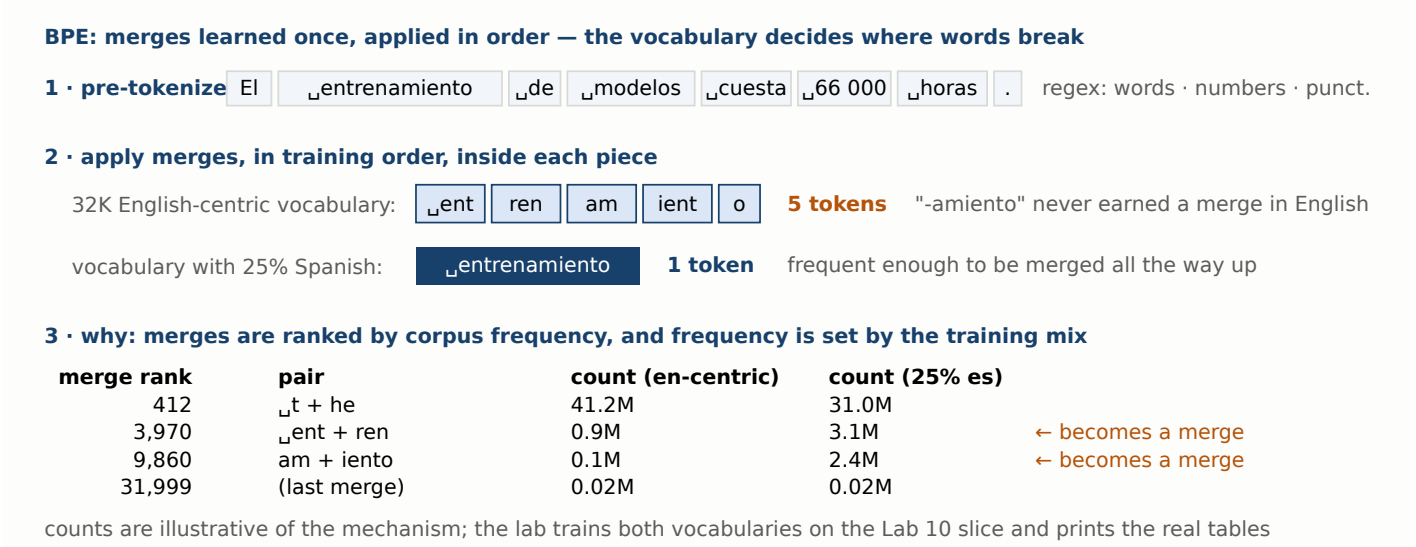


Figure 12.2 — The same word, two vocabularies. Pre-tokenization fixes the pieces; the merge list, learned from pair frequencies in the training sample, decides how far each piece is rebuilt. A suffix that is common in Spanish and rare in English never earns a merge in an English-centric vocabulary, and the word pays four extra tokens for the rest of the model's life.

The algorithm, in full

BPE is two short procedures. **Training** takes a corpus and a target vocabulary size V and returns an ordered list of merges. Pre-tokenize the corpus into pieces (words with their leading space, numbers, punctuation) and count each distinct piece once with its frequency, since identical pieces behave identically. Represent each piece as a sequence of base symbols — the 256 byte values in byte-level BPE, so the initial vocabulary is 256 and nothing is ever out-of-vocabulary. Then repeat until the vocabulary has V entries: count every adjacent pair of symbols across all pieces, weighting each pair by its piece's frequency; take the most frequent pair (a, b); append the merge $a + b \to ab$ to the list and add ab to the vocabulary; and replace every occurrence of the adjacent pair (a, b) with ab in every piece. Each iteration adds exactly one token, so training runs $V - 256$ iterations; efficient implementations keep the pair counts incrementally and update only the pieces a merge touched, which is what makes a 128K vocabulary on billions of words tractable. **Encoding** new text replays the list: pre-tokenize, split each piece into bytes, and apply the merges *in training order*, each one wherever its pair occurs; the symbols left at the end are the tokens, looked up as ids. The order is what makes the tokenizer deterministic — the same text always produces the same tokens — and it is why a merge learned late can never fire before an earlier merge that overlaps it.

```
# BPE from scratch: training returns an ordered merge list; encoding replays it
import collections, re
PAT = re.compile(r" ?[^\W\d_]+| ?\d+| ?[^\w\s]+|\s+")  # GPT-2-style pieces
def pretokenize(text): return PAT.findall(text)
def merge_pair(sym, a, b):  # replace every adjacent (a, b) by a+b
    out, i = [], 0
    while i < len(sym):
        if i < len(sym) - 1 and sym[i] == a and sym[i + 1] == b:
            out.append(a + b); i += 2
        else: out.append(sym[i]); i += 1
    return tuple(out)
def train_bpe(corpus, vocab_size):
```

```
    pieces = collections.Counter(tuple(bytes([c]) for c in w.encode())
                                 for w in pretokenize(corpus))
    n_vocab, merges = 256, []            # 256 byte symbols: nothing is OOV
    while n_vocab < vocab_size:
        pairs = collections.Counter()
        for sym, n in pieces.items():
            for a, b in zip(sym, sym[1:]): pairs[(a, b)] += n  # × freq
        if not pairs: break
        a, b = max(pairs, key=pairs.get)   # most frequent adjacent pair
        merges.append((a, b)); n_vocab += 1
        pieces = collections.Counter({merge_pair(s, a, b): n
                                      for s, n in pieces.items()})
    return merges                        # the tokenizer IS this ordered list
def encode(text, merges):
    toks = []
    for w in pretokenize(text):
        sym = tuple(bytes([c]) for c in w.encode())
        for a, b in merges: sym = merge_pair(sym, a, b)   # replay, in order
        toks.extend(sym)
    return toks
```

A trace makes the order concrete. Train on a four-sentence Spanish corpus — *el entrenamiento de modelos cuesta horas. el rendimiento del entrenamiento mejora. los modelos aprenden del entrenamiento. el conocimiento y el rendimiento crecen.* — for twelve merges. Table 12.0 is the real output of the code above (ties broken by first occurrence): each merge with the pair count that won it, in order.

Table 12.0 — Twelve merges on a toy Spanish corpus. Computed with the code above; ␣ marks a leading space, which is part of the symbol. After twelve merges ␣*entrenamiento* is a single token; `encode("el entrenamiento cuesta")` returns `el · ␣entrenamiento · ␣ · c · u · e · s · t · a` — the word that appeared three times is one token, the word that appeared once is still bytes.

#	PAIR	→ NEW SYMBOL	COUNT	WHAT IT BUILDS
1	e + n	en	17	the most frequent bigram in the corpus
2	en + t	ent	9	merges compose: uses merge 1
3	e + l	el	8	*el*, *del*, *modelos*
4	r + en	ren	6	shared by *entrenamiento*, *rendimiento*, *aprenden*, *crecen*
5	m + i	mi	6	from *-miento* ×6
6	mi + ent	mient	6	uses merges 2 and 5
7	mient + o	miento	6	the suffix is one symbol after seven steps

#	PAIR	→ NEW SYMBOL	COUNT	WHAT IT BUILDS
8	d + el	del	4	word-internal; no leading space
9	␣ + ent	␣ent	3	the stem of *entrenamiento* starts, with its space
10	␣ent + ren	␣entren	3	uses merge 4
11	␣entren + a	␣entrena	3	
12	␣entrena + miento	␣entrenamiento	3	one token for the three-times word

Three things in the trace generalize to a 128K vocabulary on a trillion words. The merge order is frequency order, so what a language gets is exactly proportional to how often its pieces occur in the *tokenizer's* training sample — the mechanism behind Figure 12.2. Merges compose (6 uses 2 and 5; 12 uses 7 and 11), so a late merge can represent a long string in one token only if every prefix of it was frequent enough to be merged earlier, which is why rare long words stay fragmented even in huge vocabularies. And a leading space is part of the symbol (merge 9 builds ␣*ent* from the space, merge 8 builds *del* without one), which is why "el" after a space and "el" inside *modelos* are different tokens with different embeddings — one of several reasons a model's arithmetic and spelling depend on pre-tokenization details that look cosmetic.

Two consequences follow from the mechanism and explain most of what you will observe. First, **the tax is set by the tokenizer's training mix, not the model's.** A vocabulary trained on 90% English will spend its merges on English pairs, and no amount of Spanish in the *model's* pre-training corpus changes the merge list afterwards; this is why the tokenizer decision has to be made with Chapter 10's mix in hand, and why the mix must be re-measured in tokens once it is made. Second, **vocabulary size trades fertility against learnability.** More merges mean longer tokens and fewer of them, but each new token is a row in the embedding matrix that must be seen often enough to be learned; a token that appears a hundred times in a trillion-token corpus is an undertrained row that the model handles worse than the fragments it replaced. The sweet spot moves with the amount of training, which is the subject of the next section.

```
# train a byte-level BPE on the Lab 10 slice with a chosen language mix
(tokenizers library)
from tokenizers import Tokenizer, models, trainers, pre_tokenizers, decoders
def train_bpe(texts, vocab_size=32_000):
    tok = Tokenizer(models.BPE(byte_fallback=True))
    tok.pre_tokenizer = pre_tokenizers.Sequence([
        pre_tokenizers.Split(pattern=r"\d{1,3}", behavior="isolated"),  # Llama-3
digit groups
        pre_tokenizers.ByteLevel(add_prefix_space=False)])            # GPT-2
regex underneath
    tok.decoder = decoders.ByteLevel()
    trainer = trainers.BpeTrainer(vocab_size=vocab_size, show_progress=False,

initial_alphabet=pre_tokenizers.ByteLevel.alphabet())
```

```
    tok.train_from_iterator(texts, trainer); return tok
mix_en  = train_bpe(sample(en_docs, 200_000))                          # English-
centric
mix_lat = train_bpe(sample(en_docs, 120_000) + sample(es_docs, 50_000)
                    + sample(pt_docs, 30_000))                         # 25% es /
15% pt
for name, t in (("en-centric", mix_en), ("25% es / 15% pt", mix_lat)):
    print(name, {lang: np.mean([len(t.encode(d).ids) / len(d.split()) for d in
test[lang]])
                 for lang in ("en", "es", "pt")})                  # fertility
per language
```

12.3 How big should the vocabulary be?

Here is the question the field answered in 2024 and most models still get wrong. For years vocabulary size was set by convention — 32K for Llama 2 and the models that copied it, 50K for GPT-2's descendants — while every other dimension of the model was set by scaling laws. The study that applied the scaling-law method to V trained models from 33M to 3B parameters on up to 500B characters, fit the loss as a function of non-embedding parameters, vocabulary parameters, and training data, and found by three independent routes (IsoFLOP curves, a derivative argument, and a parametric fit) the same answer: *the compute-optimal vocabulary grows with the compute budget*, and the models of 2024 were under-vocabularied by a wide margin — Llama 2 70B, by the fit, should have had at least 216K tokens rather than 32K[3]. The experiment that makes it concrete: at 2.3×10^{21} FLOPs, raising a 3B model's vocabulary from 32K to 43K — and shrinking the rest of the model to keep FLOPs equal — improved ARC-Challenge from 29.1 to 32.0[3].

The intuition is a budget split. Vocabulary parameters $N_V = Vd$ buy compression — fewer tokens per document, so more documents per FLOP — while non-vocabulary parameters N_{nv} buy capacity. The loss, in the parametric form the study fits, is

$$L(N_{nv}, N_V, H) \;=\; E \;+\; \frac{A_1}{N_{nv}^{\alpha}} \;+\; \frac{A_2}{N_V^{\beta}} \;+\; \frac{A_3}{H^{\gamma}},$$

where H is the training data measured in *characters* rather than tokens (so that two vocabularies can be compared on the same text), and $A_i, \alpha, \beta, \gamma, E$ are fitted. Measuring data in characters is the trick that makes the question well-posed: a bigger vocabulary sees fewer tokens of the same characters, spends less compute on them, and can reinvest the savings. Minimizing the loss at fixed FLOPs gives an optimal V that rises as a power of compute; the fitted exponent is below one — vocabulary parameters should grow more slowly than the rest of the model, but they should grow — which is why the 7B models of 2025 settled on 128K–150K and Gemma 3 on 262K, and why a 1B model trained for a trillion tokens still wants more than 32K[3]. Figure 12.3 draws the law with the shipped models on it.

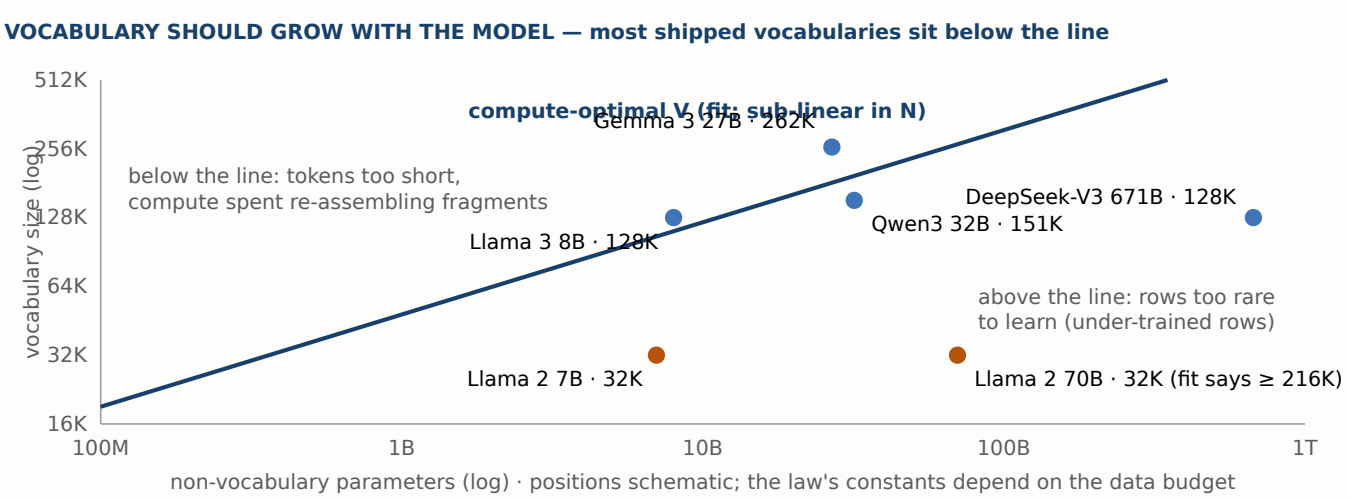


Figure 12.3 — Vocabulary size against model size. The line is the shape of the fitted compute-optimal vocabulary (sub-linear in non-vocabulary parameters); the points are shipped choices. The 2024–2025 generation moved up by 4–8×; the fit says the largest models should go further still. Positions are schematic — the law's constants depend on the training-data budget.

Two caveats keep the law honest. The study measures English-dominant corpora; for a multilingual mix the optimal V is larger still, because the merges must cover several languages' frequent pieces — a 128K vocabulary shared across 119 languages (Qwen3) gives each language the coverage of a much smaller one. And the law assumes the vocabulary is *trained on the right mix*: 262K English-centric tokens do nothing for Spanish that 32K did not. Size and mix are two decisions, and the second is the one that moves the tax.

12.4 The architecture bill: embeddings, tying, and the softmax

A vocabulary is not only a compression scheme; it is two matrices and the most expensive single operation in the forward pass. The input embedding is $V \times d$ and the output projection (the "unembedding" that produces logits) is $d \times V$; untied, that is $2Vd$ parameters, and for the 2024–2025 vocabularies the number is no longer small. Llama 3 8B, with $V = 128{,}256$ and $d = 4{,}096$, carries about 525M parameters in each — 1.05B of its 8.03B, 13% of the model, in two lookup tables[4]. Scale the model down and the share goes up, because d shrinks while V does not: a 1B model with $d = 2{,}048$ and the same vocabulary has 262M embedding parameters per matrix, and the small models of 2025 (Llama 3.2 1B, Gemma 3 1B, Qwen3 0.6B) all tie the two matrices and still spend 20–30% of their parameters on the vocabulary (figures computed from the released configurations). The output projection also costs compute on every token:

$$\text{FLOPs}_{\text{softmax}} = 2\,Vd \;\; \text{per token}, \qquad \frac{\text{FLOPs}_{\text{softmax}}}{2N} = \frac{Vd}{N},$$

which is $128\text{K} \times 4096/8\text{B} \approx 6.5\%$ for Llama 3 8B and about 21% for a 1B model with $d = 2048$;

so for a small model one token in five of compute goes to deciding among 128K candidates, most of which are never the answer. Chapter 8's FLOP accounting folds this into the $2N$ rule; it is worth separating here because it is the term that vocabulary size moves directly, and the reason the scaling law of §12.3 has an optimum rather than a monotone "bigger is better."

Tied or untied? Weight tying — using one matrix for both input and output — halves the vocabulary parameters and was the default of the small-model era; its justification is that a token's input representation and its output representation should be the same vector, which is approximately true and saves Vd parameters that a small model can spend elsewhere[5]. Large models untie, because at $d = 4{,}096$ and up the two roles diverge (the input embedding is a lookup, the output projection is a classifier over contexts) and the parameters are cheap relative to the rest. The practical rule from the shipped configurations is a threshold on Vd/N: below roughly 3B parameters tie, above untie, and when in doubt run the ablation on a 300M proxy — it is a one-line config change. Figure 12.4 plots the two bills against vocabulary size.

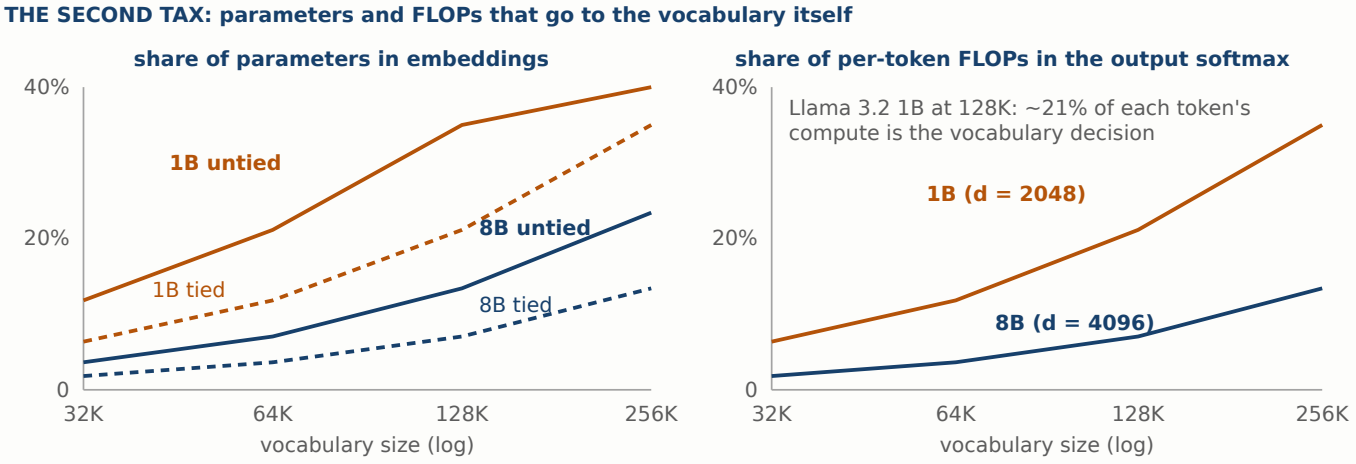


Figure 12.4 — What a vocabulary costs the architecture. Both panels are computed from Vd against a fixed non-vocabulary parameter count (1B and 7B). Small models feel the vocabulary twice — in parameters they cannot spend elsewhere and in compute spent on the output projection — which is why the §12.3 law has an optimum and why small models tie their embeddings.

One more architectural knot ties to the tokenizer and is worth untangling here rather than in a chapter of its own. The embedding dimension d is set by the model's width, but the *effective* dimension available to each token is what matters for rare rows: a token seen a thousand times in training gets a thousand gradient updates to its d numbers, and a token seen ten times gets ten — so a large vocabulary with a long tail of rare tokens is partly dead weight regardless of d. The fix the 2025 models converge on is not a smaller vocabulary but a *better-trained* one: a vocabulary trained on the same mix as the model, so that every row is common in the data the model sees, and (for the tail) initialization strategies that give rare rows a sensible starting point, which is the subject of adaptation.

12.5 Adapting a vocabulary: extend, swap, or train from scratch

Most teams do not train a tokenizer and a model from nothing; they start from an open checkpoint whose vocabulary was chosen for someone else's mix. Three routes lower the tax, in increasing order of cost and payoff, and the choice between them is the decision the chapter's flowchart ends on.

Extend. Keep the base vocabulary and append tokens for the target languages — the recipe Chinese-LLaMA used to add about 20K Chinese tokens to Llama's 32K, cutting Chinese fertility by roughly half, then continued pre-training so that the new rows learned[6]. The new rows need an initialization that is not random, or the continued pre-training spends

its first billions of tokens teaching them to be plausible vectors; the standard trick (FOCUS and its relatives) initializes a new token as a weighted combination of the base-vocabulary tokens it overlaps with or is semantically near, which measurably shortens the recovery[7]. Extension preserves everything the base model knows about English and costs a continued-pre-training run on the target languages — tens of billions of tokens for a 7B model — which Chapter 20 budgets. Its ceiling is the base pre-tokenizer and merge order: appended tokens are applied as a vocabulary lookup after the base merges, so they help whole words and do little for morphology that the base already fragmented.

Swap. Replace the tokenizer entirely with one trained on your mix, map the old embedding matrix onto the new vocabulary (exact matches copied, the rest initialized from overlapping subwords as above, or predicted by a small hypernetwork trained for the purpose — the zero-shot tokenizer transfer method[8]), and continue pre-training. The 2024 study that did this systematically for code found that a swapped tokenizer with a larger vocabulary and a better pre-tokenizer recovered the base model's quality after fine-tuning on about 50B tokens and then *beat* it on compression and downstream tasks — the gains from a better vocabulary outlast the cost of the swap[9]. Swapping is the route when the base vocabulary is wrong in its pre-tokenization (digit handling, whitespace, scripts), which extension cannot repair.

Train from scratch. If you are pre-training from random initialization anyway, the tokenizer is free: train it on a sample of the Chapter 10 mix, at the V the §12.3 law suggests for your compute, with the pre-tokenization choices you want (digit groups, whitespace policy, byte fallback), and re-measure the mix in tokens before training. This is the only route in which the tax is set by you rather than inherited, and it is what every frontier model in Table 12.1 did.

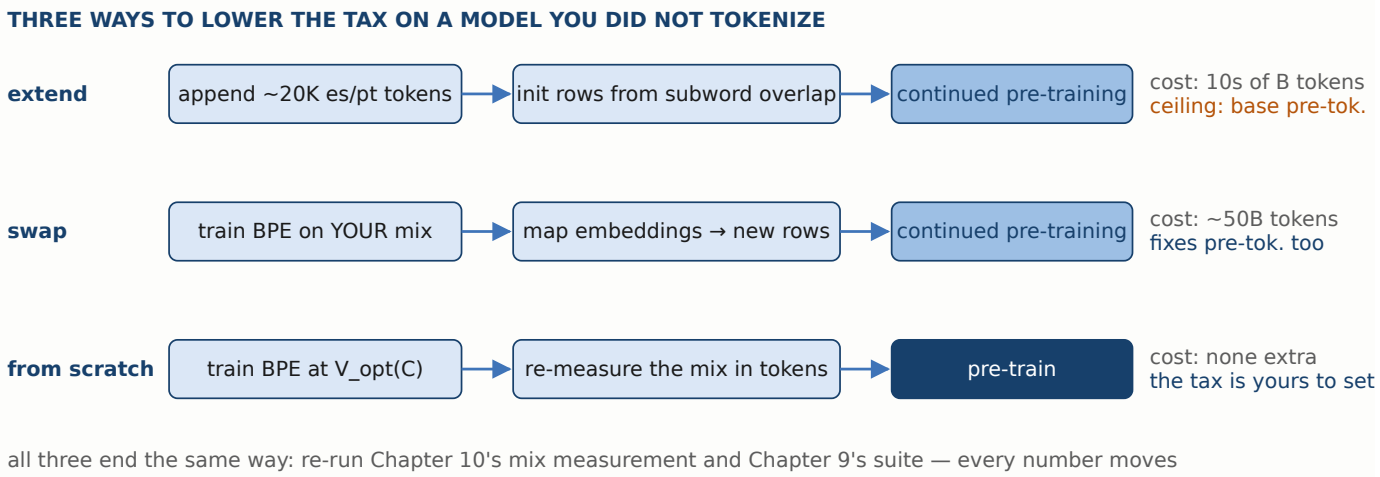


Figure 12.5 — Extend, swap, or start over. Extension is cheap and bounded by the base pre-tokenizer; swapping costs a recovery run and can fix everything; training from scratch is free when you are pre-training anyway. Each ends with the same two measurements.

```
# extend a vocabulary; initialize new rows from overlapping subwords (FOCUS-style,
simplified)
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
base_tok = AutoTokenizer.from_pretrained(BASE)
model = AutoModelForCausalLM.from_pretrained(BASE)
new_tokens = [t for t in es_pt_bpe.get_vocab() if t not in base_tok.get_vocab()]
[:20_000]
base_tok.add_tokens(new_tokens)                                  # appended after the
```

```
base merges
old_n = model.get_input_embeddings().weight.shape[0]
model.resize_token_embeddings(len(base_tok))
E = model.get_input_embeddings().weight.data              # (V_old + 20k, d)
with torch.no_grad():
    for i, t in enumerate(new_tokens):
        pieces = base_tok(t.replace("Ġ", " "), add_special_tokens=False).input_ids
# base split
        E[old_n + i] = E[pieces].mean(0)          # mean of the fragments it
replaces
    if model.config.tie_word_embeddings is False:
        model.get_output_embeddings().weight.data[old_n:] = E[old_n:]
# then: continued pre-training on the es/pt mix (Ch. 20); re-measure fertility
(§12.1)
```

12.6 Beyond tokens: superwords and bytes

Two 2025 directions attack the tax from opposite ends, and both are worth knowing because one of them is already shipping. **SuperBPE** keeps the architecture and changes the tokenizer's one restriction: after the ordinary subword stage, a second stage lifts the whitespace boundary and lets frequent multi-word expressions become single tokens, so "of the", "in order to", "New York" each cost one. At the same 200K vocabulary the encoding is up to 33% shorter, and the 8B model trained with it — same parameters, same compute, only the tokenizer changed — gained 4.0 points across 30 tasks and 8.2 on MMLU while needing 27% less inference compute, because every sequence is shorter[2]. For a Spanish-first model the same mechanism captures the clitics, articles, and fixed phrases that the Romance languages spend tokens on ("de la", "en el", "a través de"), which is the kind of tax a subword vocabulary can never remove.

Byte-level models remove the vocabulary altogether. The Byte Latent Transformer reads raw bytes and groups them into variable-length *patches* whose boundaries are set by the entropy of the next byte — predictable stretches become long patches handled by a small local model, surprising ones get short patches and the full model's attention — so compute is allocated by difficulty rather than by a fixed merge table. At 8B parameters and 4T bytes it matched the tokenized Llama 3 recipe at the same FLOPs, with better robustness to noise and spelling and a new axis to scale (patch size) that lets inference FLOPs fall as the model grows[10]. For languages the tokenizer never saw, bytes are the only scheme with no tax at all — a Quechua word costs its bytes, not the fragments an English vocabulary makes of it — which is why byte-level is the direction to watch for exactly the languages this book cares about. It is not yet the production default: the infrastructure (packing, inference kernels, serving) assumes tokens, and the entropy model adds a stage; but a team starting a from-scratch model in 2026 should at least run the comparison on a small proxy.

12.7 What the big labs do

Table 12.1 — Tokenizers, as shipped. Vocabulary size, scheme, and the choices that set the tax.

MODEL	V	SCHEME AND PRE-TOKENIZATION	WHAT IT SAYS ABOUT THE TAX
Llama 2[11]	32K	SentencePiece BPE; digits split individually; byte fallback	The English-centric baseline; 1.4–1.7× fertility for es/pt, the vocabulary the §12.3 law says was 7× too small at 70B
Llama 3[4]	128K	tiktoken-style byte-level BPE: 100K from the GPT-4 vocabulary plus 28K tokens for non-English; digits in groups of up to 3	Better compression than Llama 2 on English and a large cut for other languages; untied 525M-row embeddings at 8B
Qwen3[12]	151,669	Byte-level BPE shared across 119 languages	One vocabulary for the whole mix; per-language coverage thinner than the size suggests
Gemma 3[13]	262K	SentencePiece with split digits, preserved whitespace, byte fallback; same as Gemini	The largest shipped vocabulary; 20–30% of the 1B model's parameters are embeddings (tied)
DeepSeek-V3[14]	128K	Byte-level BPE; pre-tokenizer changed to merge punctuation with line breaks; token-boundary bias mitigated by random splitting in training	Pre-tokenization is a design surface, not a default
SuperBPE (research)[2]	200K	Subword stage then superword stage across whitespace	Up to 33% fewer tokens; +4.0 avg / +8.2 MMLU at 8B; 27% less inference compute
BLT (research)[10]	256 bytes	Entropy-based dynamic patches; no vocabulary	Matches Llama 3 at 8B/4T bytes; no tax for unseen languages

12.8 Practical considerations

Measure before you choose, on your corpus. The §12.1 script over a few thousand documents per language, for every tokenizer you are considering, is an hour of CPU and tells you the tax to the percent. Put the table in the dataset card next to Chapter 10's mix, and re-state the mix in tokens: the number that goes into DoReMi and the stage schedule is the token share, not the document share.

Train the tokenizer on the mix you will train the model on, with the same language weights, and on a sample large enough that the Spanish and Portuguese merges are learned from Spanish and Portuguese (hundreds of thousands of documents per language). A tokenizer trained on an English sample with the target languages "added later" is the extend route with extra steps.

Watch the tail of the vocabulary. After training, count how often each token appears in a held-out sample of the mix; rows that appear fewer than a few hundred times in the whole pre-training run are undertrained and cost more than they save. Prune them (re-train with a smaller V) or accept them knowingly. The same count, by language, is the fairness audit: it tells you whether the vocabulary's budget went where the mix said it should.

Pre-tokenization choices are permanent. Digit grouping, whitespace handling, how punctuation attaches to words, whether code indentation is preserved — each is a regex decision made before BPE and impossible to change without a swap. Decide them with the use cases in front of you: three-digit groups for arithmetic, preserved whitespace runs for code, and byte fallback always.

Benchmarks move when the tokenizer moves. A score on a Spanish benchmark after a tokenizer change is a different measurement, not a better model, until the model has been re-trained on the new tokens. Chapter 9's noise bars and Chapter 11's contamination tests apply to the new tokenizer as to a new model.

Common pitfalls

Reporting the data mix in bytes or documents.

Spanish at 4.6% of documents is a different share of tokens under every vocabulary. Every mix decision downstream of Chapter 10 is in tokens.

Inheriting a 32K vocabulary because the base model has one.

The scaling law says it is too small at any budget above a few 10^{20} FLOPs, and extension cannot fix the pre-tokenizer.

Extending with random-initialized rows.

The first billions of continued-pre-training tokens go to making them plausible; initialize from overlapping subwords.

Choosing V by the law and training the tokenizer on the wrong mix.

Size without mix buys nothing for the target languages.

Forgetting the architecture bill at small scale.

A 128K vocabulary on a 500M model is a third of the parameters and a fifth of the FLOPs; tie the embeddings and consider a smaller V.

Changing the tokenizer mid-project without re-running everything.

Fertility, mix, context budget, benchmark numbers, and the data-constrained epoch count all change at once.

12.9 Decision framework

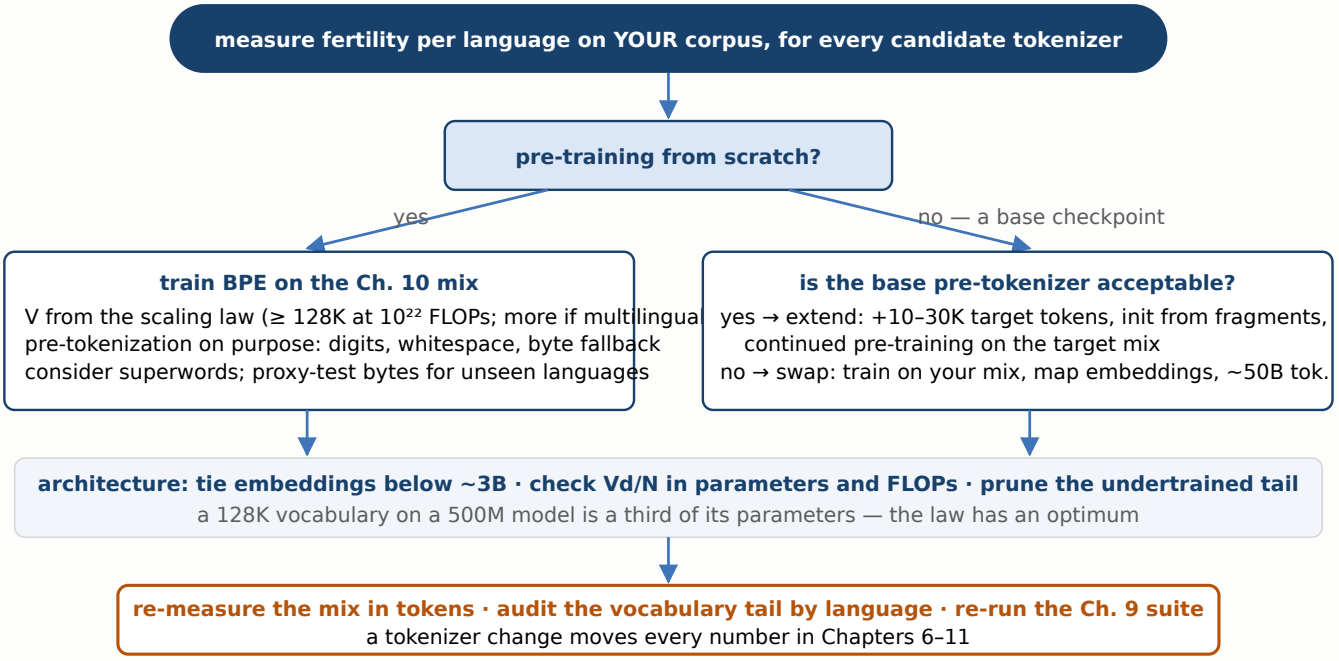


Figure 12.6 — The tokenizer decision. Measure first; the route depends on whether you are pre-training from scratch and on whether the base pre-tokenizer can be lived with; the architecture bill and the three re-measurements apply to every route.

When it hurts. A larger vocabulary hurts a small model (the §12.4 bill) and a thin corpus (undertrained rows); extension hurts when the base pre-tokenizer is the problem; a swap hurts when the recovery budget is not there — 50B tokens is a real run — and training from scratch with a mix-trained tokenizer hurts only when the mix itself is wrong, which is Chapter 10's problem, not this chapter's.

How this scales. The optimal vocabulary grows with compute, so the right V for your next model is larger than for this one; the architecture bill shrinks as d grows, so large models can afford the vocabulary small ones cannot; and the tax on under-served languages does not shrink with scale at all unless the tokenizer's training mix changes — which is the one lever this chapter asks you to pull on purpose.

12.10 Summary

A vocabulary sets the price of every token in every language, and the price is paid in compute, context, and the share of the data mix. Measure it as fertility per language on your own corpus, for every tokenizer you consider — the Romance languages typically cost 1.2–1.7× English depending on the vocabulary's generation, and languages the tokenizer never saw cost far more. The mechanism is BPE's merge list, learned from pair frequencies in the tokenizer's training sample, which is why the tax is set by that sample's mix and not by the model's; the size is a scaling-law decision — the compute-optimal vocabulary grows with compute, and most shipped models are below the line — and the architecture pays for it twice, in embedding parameters (13% of Llama 3 8B, 20–30% of the 1B models) and in the output softmax (a fifth of a small model's FLOPs). On a base checkpoint, extend when the pre-tokenizer is acceptable and swap when it

is not, initializing new rows from the fragments they replace and budgeting the recovery; from scratch, train the tokenizer on the Chapter 10 mix at the law's V with pre-tokenization chosen on purpose, and look at superwords and bytes before defaulting to subwords. Whatever the route, re-measure the mix in tokens, audit the vocabulary's tail by language, and re-run the suite — the chapters before this one gave their numbers in documents and bytes, and from here on the book counts in tokens.

References

1. Petrov, A., La Malfa, E., Torr, P., & Bibi, A. (2023). Language Model Tokenizers Introduce Unfairness Between Languages. *NeurIPS*. arXiv:2305.15425. Up to 15× tokenized-length disparity across languages; >4× even for byte/character schemes; cost, latency, and context consequences. Fertility and parity metrics follow Rust, P., et al. (2021). How Good is Your Tokenizer? *ACL*. arXiv:2012.15613.
2. Liu, A., et al. (2025). SuperBPE: Space Travel for Language Models. arXiv:2503.13423. Subword-then-superword BPE; up to 33% fewer tokens at 200K; 8B model +4.0 average over 30 tasks, +8.2 MMLU, 27% less inference compute at fixed parameters and training FLOPs.
3. Tao, C., et al. (2024). Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies. *NeurIPS*. arXiv:2407.13623. Models 33M–3B on up to 500B characters; three estimation methods agree the optimal vocabulary grows with compute; Llama 2 70B predicted ≥ 216K vs. 32K; 3B model at 2.3×10^{21} FLOPs: 32K → 43K raises ARC-Challenge 29.1 → 32.0.
4. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. 128K-token tiktoken-based vocabulary (100K + 28K non-English); improved compression over Llama 2; embedding sizes from the released configurations.
5. Press, O., & Wolf, L. (2017). Using the Output Embedding to Improve Language Models. *EACL*. arXiv:1608.05859. Weight tying between input and output embeddings.
6. Cui, Y., Yang, Z., & Yao, X. (2023). Efficient and Effective Text Encoding for Chinese LLaMA and Alpaca. arXiv:2304.08177. ~20K-token vocabulary extension of Llama; roughly halved Chinese token counts; continued pre-training.
7. Dobler, K., & de Melo, G. (2023). FOCUS: Effective Embedding Initialization for Monolingual Specialization of Multilingual Models. *EMNLP*. arXiv:2305.14481. Initializing new-token embeddings from semantically similar overlapping tokens.
8. Minixhofer, B., Ponti, E. M., & Vulić, I. (2024). Zero-Shot Tokenizer Transfer. *NeurIPS*. arXiv:2405.07883. A hypernetwork predicts embeddings for an arbitrary new tokenizer; swap without retraining from scratch.
9. Dagan, G., Synnaeve, G., & Rozière, B. (2024). Getting the Most Out of Your Tokenizer for Pre-training and Domain Adaptation. *ICML*. arXiv:2402.01035. Tokenizer size, pre-tokenization, and training data for code; swapping a pre-trained model's tokenizer recovers with ~50B tokens of fine-tuning and improves compression and downstream results.
10. Pagnoni, A., et al. (2024). Byte Latent Transformer: Patches Scale Better Than Tokens. Meta FAIR. arXiv:2412.09871. Entropy-based dynamic byte patches; FLOP-controlled scaling to 8B parameters and 4T bytes

matching tokenized models; patch size as a new scaling axis.

11. Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288. 32K SentencePiece BPE; individual-digit splitting; byte fallback.
12. Qwen Team (2025). Qwen3 Technical Report. arXiv:2505.09388. 151,669-token byte-level BPE across 119 languages.
13. Gemma Team (2025). Gemma 3 Technical Report. arXiv:2503.19786. 262K SentencePiece vocabulary shared with Gemini; split digits, preserved whitespace, byte fallback.
14. DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437. 128K byte-level BPE; pre-tokenizer modified for punctuation and line breaks; random splitting of combined tokens during training to reduce boundary bias.